

KEMELLY LUANE MOURA PEREIRA

TURMA: EIEL – 21A

VETORES E LISTAS - LLP

Parte II – Exercícios com Vetores 9. Preenchimento e exibição Crie um programa em C++ que receba 10 números inteiros, armazene-os em um vetor e depois apresente todos os valores armazenados. Espaço para algoritmo/código:

```
#include <iostream>

using namespace std;

int main() {

    // Declaração de um vetor do tipo inteiro para armazenar 10 elementos
    int numeros[10];

    // Loop para leitura e preenchimento do vetor
    cout << "=== Leitura de Dados ===" << endl;
    for (int i = 0; i < 10; i++) {
        cout << "Digite o " << i + 1 << "º numero inteiro: ";
        cin >> numeros[i];
    }

    // Loop para exibição dos valores armazenados
    cout << "\n=== Valores Armazenados no Vetor ===" << endl;
    for (int i = 0; i < 10; i++) {
        cout << "Posicao [" << i << "]: " << numeros[i] << endl;
    }
}
```

```
    return 0;
}
```

10. Maior e menor valor Utilizando um vetor com 10 números inteiros, determine o maior valor, o menor valor, a posição do maior valor e a posição do menor valor.

Espaço para algoritmo/código:

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int numeros[10];
```

```
    // Leitura dos 10 números inteiros
```

```
    cout << "=== Leitura dos Elementos ===" << endl;
```

```
    for (int i = 0; i < 10; i++) {
```

```
        cout << "Digite o " << i + 1 << "º numero: ";
```

```
        cin >> numeros[i];
```

```
    }
```

```
    // Inicializa o maior e menor valor com o primeiro elemento do vetor
```

```
    int maior = numeros[0];
```

```
    int menor = numeros[0];
```

```
    int posMaior = 0;
```

```
    int posMenor = 0;
```

```
    // Percorre o vetor a partir do segundo elemento para comparar
```

```
    for (int i = 1; i < 10; i++) {
```

```
        if (numeros[i] > maior) {
```

```
            maior = numeros[i];
```

```
            posMaior = i;
```

```

    }

    if (numeros[i] < menor) {
        menor = numeros[i];
        posMenor = i;
    }
}

// Exibição dos resultados obtidos

cout << "\n=== Resultados ===" << endl;

cout << "Maior valor: " << maior << " (Posicao/Index: [" << posMaior << "])" << endl;

cout << "Menor valor: " << menor << " (Posicao/Index: [" << posMenor << "])" << endl;

return 0;
}

```

11. Média das notas Desenvolva um programa que armazene as notas de 5 alunos em um vetor e apresente: todas as notas, a média da turma, a maior nota, a menor nota e a quantidade de notas acima da média. Espaço para algoritmo/código:

```

#include <iostream>

using namespace std;

int main() {
    double notas[5];
    double soma = 0.0;

    // Leitura das notas e cálculo da soma total

    cout << "=== Leitura das Notas ===" << endl;

    for (int i = 0; i < 5; i++) {
        cout << "Digite a nota do " << i + 1 << "º aluno: ";
    }
}

```

```

        cin >> notas[i];

        soma += notas[i];
    }

    // Cálculo da média da turma
    double media = soma / 5.0;

    // Inicialização para maior e menor nota
    double maior = notas[0];
    double menor = notas[0];
    int acimaDaMedia = 0;

    // Verificação de maior, menor e contagem de notas acima da média
    for (int i = 0; i < 5; i++) {
        if (notas[i] > maior) {
            maior = notas[i];
        }
        if (notas[i] < menor) {
            menor = notas[i];
        }
        if (notas[i] > media) {
            acimaDaMedia++;
        }
    }

    // Exibição de todas as notas armazenadas
    cout << "\n=== Relatorio da Turma ===" << endl;
    cout << "Notas registradas: ";
    for (int i = 0; i < 5; i++) {

```

```

        cout << "[" << notas[i] << "]" ";
    }

    cout << endl;

    // Exibição dos dados estatísticos

    cout << "\nMedia da turma: " << media << endl;

    cout << "Maior nota: " << maior << endl;

    cout << "Menor nota: " << menor << endl;

    cout << "Quantidade de alunos com nota acima da media: " << acimaDaMedia <<
endl;

    return 0;
}

```

12. Pares e ímpares Leia 10 números inteiros e armazene-os em um vetor. Ao final, informe a quantidade de números pares e a quantidade de números ímpares. Espaço para algoritmo/código:

```
#include <iostream>
```

```
using namespace std;
```

```

int main() {

    double notas[5];

    double soma = 0.0;

    // Leitura das notas e cálculo da soma total

    cout << "=== Leitura das Notas ===" << endl;

    for (int i = 0; i < 5; i++) {

        cout << "Digite a nota do " << i + 1 << "º aluno: ";

        cin >> notas[i];

        soma += notas[i];
    }
}

```

```
}
```

```
// Cálculo da média da turma
```

```
double media = soma / 5.0;
```

```
// Inicialização para maior e menor nota
```

```
double maior = notas[0];
```

```
double menor = notas[0];
```

```
int acimaDaMedia = 0;
```

```
// Verificação de maior, menor e contagem de notas acima da média
```

```
for (int i = 0; i < 5; i++) {
```

```
    if (notas[i] > maior) {
```

```
        maior = notas[i];
```

```
    }
```

```
    if (notas[i] < menor) {
```

```
        menor = notas[i];
```

```
    }
```

```
    if (notas[i] > media) {
```

```
        acimaDaMedia++;
```

```
    }
```

```
}
```

```
// Exibição de todas as notas armazenadas
```

```
cout << "\n=== Relatorio da Turma ===" << endl;
```

```
cout << "Notas registradas: ";
```

```
for (int i = 0; i < 5; i++) {
```

```
    cout << "[" << notas[i] << "] ";
```

```
}
```

```

cout << endl;

// Exibição dos dados estatísticos

cout << "\nMedia da turma: " << media << endl;

cout << "Maior nota: " << maior << endl;

cout << "Menor nota: " << menor << endl;

cout << "Quantidade de alunos com nota acima da media: " << acimaDaMedia <<
endl;

return 0;
}

```

13. Busca em vetor Leia 10 números e depois solicite ao usuário um número para procurar. Caso o valor seja encontrado, informe sua posição. Caso contrário, apresente a mensagem “Valor não encontrado”. Espaço para algoritmo/código:

```

#include <iostream>

using namespace std;

int main() {
    int numeros[10];

    // Leitura dos 10 números inteiros para preencher o vetor
    cout << "=== Leitura dos Elementos ===" << endl;
    for (int i = 0; i < 10; i++) {
        cout << "Digite o " << i + 1 << "º numero: ";
        cin >> numeros[i];
    }
}

```

```

    }

    // Leitura do número a ser pesquisado
    int valorBusca;

    cout << "\nDigite o numero que deseja procurar: ";
    cin >> valorBusca;

    // Busca linear no vetor
    bool encontrado = false;

    for (int i = 0; i < 10; i++) {
        if (numeros[i] == valorBusca) {
            cout << "Valor " << valorBusca << " encontrado na posicao (indice): [" << i <<
            "]" << endl;
            encontrado = true;
        }
    }

    // Caso o número não tenha sido localizado em nenhuma posição
    if (!encontrado) {
        cout << "Valor nao encontrado." << endl;
    }

    return 0;
}

```

Parte III – Exercícios com Listas Encadeadas 14. Identificando os elementos Considere a lista: $10 \rightarrow 25 \rightarrow 30 \rightarrow 45 \rightarrow \text{NULL}$. Responda: a) Qual é o primeiro elemento? b)

Qual é o último elemento? c) Qual elemento vem depois do 25? d) Quantos nós existem? e) Para onde aponta o nó que contém o valor 45? Resposta / código

```
#include <iostream>
```

```
using namespace std;
```

```
// Definição da estrutura de um Nó
```

```
struct No {
```

```
    int valor;
```

```
    No* proximo;
```

```
// Construtor para compatibilidade com versões anteriores
```

```
No(int v, No* p = NULL) {
```

```
    valor = v;
```

```
    proximo = p;
```

```
}
```

```
};
```

```
int main() {
```

```
    // Alocação dos nós da lista: 10 -> 25 -> 30 -> 45 -> NULL
```

```
    No* no4 = new No(45, NULL);
```

```
    No* no3 = new No(30, no4);
```

```
    No* no2 = new No(25, no3);
```

```
    No* no1 = new No(10, no2);
```

```
// Ponteiro 'cabeca' aponta para o primeiro elemento
```

```
No* cabeca = no1;
```

```
// Percurso e exibição da lista encadeada
```

```

cout << "=== Representacao da Lista ===" << endl;

No* atual = cabeca;

while (atual != NULL) {

    cout << atual->valor << " -> ";

    atual = atual->proximo;

}

cout << "NULL" << endl;


// Liberação da memória alocada dinamicamente

delete no1;

delete no2;

delete no3;

delete no4;


return 0;

}

```

15. Inserção em uma lista Considere a lista $10 \rightarrow 20 \rightarrow 30 \rightarrow \text{NULL}$. Represente graficamente como ela ficará depois de inserir o valor 5 no início. Em seguida, represente a inserção do valor 40 no final. Resposta / código:

```
#include <iostream>
```

```
using namespace std;
```

```
// Definição da estrutura do Nó (compatível com Dev-C++ / C++98)
```

```

struct No {

    int valor;

    No* proximo;

```

```

No(int v, No* p = NULL) {
    valor = v;
    proximo = p;
}
};

```

// Função para exibir a lista

```

void exibirLista(No* cabeca) {
    No* atual = cabeca;
    while (atual != NULL) {
        cout << atual->valor << " -> ";
        atual = atual->proximo;
    }
    cout << "NULL" << endl;
}

```

```

int main() {

```

// 1. Criação da lista inicial: 10 -> 20 -> 30 -> NULL

```

No* no3 = new No(30, NULL);

```

```

No* no2 = new No(20, no3);

```

```

No* no1 = new No(10, no2);

```

```

No* cabeca = no1;

```

```

cout << "Lista inicial:" << endl;

```

```

exibirLista(cabeca);

```

// 2. Inserção do valor 5 no início

```

No* novoInicio = new No(5, cabeca);

```

```

cabeca = novoInicio; // Atualiza o ponteiro de início para o novo nó

```

```

cout << "\nApos inserir 5 no inicio:" << endl;

exibirLista(cabeca);

// 3. Inserção do valor 40 no final

No* novoFim = new No(40, NULL);

No* atual = cabeca;

// Percorre até encontrar o último nó existente
while (atual->proximo != NULL) {
    atual = atual->proximo;
}
atual->proximo = novoFim; // Conecta o último nó ao novo nó

cout << "\nApos inserir 40 no final:" << endl;

exibirLista(cabeca);

// Liberação da memória alocada
while (cabeca != NULL) {
    No* temp = cabeca;
    cabeca = cabeca->proximo;
    delete temp;
}

return 0;
}

```

16. Remoção em uma lista Considere a lista $5 \rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow \text{NULL}$.
Mostre como ficará após remover, nesta ordem: a) 5; b) 30; c) 40. Resposta / código:

```
#include <iostream>
```

```
using namespace std;
```

```
// Definição da estrutura do Nó
```

```
struct No {
```

```
    int valor;
```

```
    No* proximo;
```

```
    No(int v, No* p = NULL) {
```

```
        valor = v;
```

```
        proximo = p;
```

```
    }
```

```
};
```

```
// Função auxiliar para exibir os elementos da lista
```

```
void exibirLista(No* cabeca) {
```

```
    No* atual = cabeca;
```

```
    while (atual != NULL) {
```

```
        cout << atual->valor << " -> ";
```

```
        atual = atual->proximo;
```

```
    }
```

```
    cout << "NULL" << endl;
```

```
}
```

```
// Função para remover um elemento com determinado valor
```

```
void removerElemento(No*& cabeca, int valor) {
```

```
if (cabeca == NULL) return; // Lista vazia
```

```
// Caso 1: O elemento a ser removido é a cabeça (início da lista)
```

```
if (cabeca->valor == valor) {  
    No* temp = cabeca;  
    cabeca = cabeca->proximo;  
    delete temp;  
    return;  
}
```

```
// Caso 2: O elemento está no meio ou no final
```

```
No* atual = cabeca;  
while (atual->proximo != NULL && atual->proximo->valor != valor) {  
    atual = atual->proximo;  
}
```

```
// Se encontrou o nó com o valor procurado
```

```
if (atual->proximo != NULL) {  
    No* temp = atual->proximo;  
    atual->proximo = temp->proximo; // Reorienta o ponteiro pulando o nó removido  
    delete temp;  
}  
}
```

```
int main() {
```

```
    // 1. Montagem da lista original: 5 -> 10 -> 20 -> 30 -> 40 -> NULL
```

```
    No* no5 = new No(40, NULL);
```

```
    No* no4 = new No(30, no5);
```

```
    No* no3 = new No(20, no4);
```

```
No* no2 = new No(10, no3);
```

```
No* no1 = new No(5, no2);
```

```
No* cabeca = no1;
```

```
cout << "Lista Original:" << endl;
```

```
exibirLista(cabeca);
```

```
// a) Remover o valor 5
```

```
removerElemento(cabeca, 5);
```

```
cout << "\na) Apos remover o valor 5:" << endl;
```

```
exibirLista(cabeca);
```

```
// b) Remover o valor 30
```

```
removerElemento(cabeca, 30);
```

```
cout << "\nb) Apos remover o valor 30:" << endl;
```

```
exibirLista(cabeca);
```

```
// c) Remover o valor 40
```

```
removerElemento(cabeca, 40);
```

```
cout << "\nc) Apos remover o valor 40:" << endl;
```

```
exibirLista(cabeca);
```

```
// Liberação da memória dos nós restantes
```

```
while (cabeca != NULL) {
```

```
    No* temp = cabeca;
```

```
    cabeca = cabeca->proximo;
```

```
    delete temp;
```

```
}
```

```
    return 0;
}
```

17. Inserção no início em C++ Utilizando a estrutura abaixo, desenvolva uma função para inserir um elemento no início da lista: struct No { int valor; No* proximo; }; Protótipo sugerido: void inserirInicio(No*& inicio, int valor); Resposta / código:

```
#include <iostream>
```

```
using namespace std;
```

```
// Estrutura do Nó fornecida no enunciado
```

```
struct No {
    int valor;
    No* proximo;
};
```

```
// Função para inserir um elemento no início da lista
```

```
void inserirInicio(No*& inicio, int valor) {
    // 1. Aloca memória dinamicamente para o novo nó
    No* novo = new No;
```

```
    // 2. Atribui o valor recebido ao novo nó
    novo->valor = valor;
```

```
    // 3. Faz o ponteiro 'proximo' do novo nó apontar para o antigo início da lista
    novo->proximo = inicio;
```

```
    // 4. Atualiza o ponteiro de início para que ele passe a apontar para o novo nó
    inicio = novo;
```

```
}
```

```
// Função auxiliar para exibir os elementos da lista
```

```
void exibirLista(No* inicio) {
```

```
    No* atual = inicio;
```

```
    while (atual != NULL) {
```

```
        cout << atual->valor << " -> ";
```

```
        atual = atual->proximo;
```

```
    }
```

```
    cout << "NULL" << endl;
```

```
}
```

```
int main() {
```

```
    No* inicio = NULL; // Inicializa a lista como vazia
```

```
    // Testando a função inserindo elementos no início
```

```
    inserirInicio(inicio, 30);
```

```
    inserirInicio(inicio, 20);
```

```
    inserirInicio(inicio, 10);
```

```
    cout << "Lista encadeada apos insercoes no inicio:" << endl;
```

```
    exibirLista(inicio); // Saída esperada: 10 -> 20 -> 30 -> NULL
```

```
    // Liberação da memória alocada
```

```
    while (inicio != NULL) {
```

```
        No* temp = inicio;
```

```
        inicio = inicio->proximo;
```

```
        delete temp;
```

```
    }
```

```
    return 0;
}
```

18. Percorrendo uma lista Desenvolva uma função em C++ que percorra uma lista encadeada e apresente todos os seus elementos. Resposta / código:

```
#include <iostream>
```

```
using namespace std;
```

```
// Estrutura do Nó
```

```
struct No {
    int valor;
    No* proximo;
};
```

```
// Função para percorrer e exibir os elementos da lista encadeada
```

```
void percorrerLista(No* inicio) {
```

```
    No* atual = inicio; // Ponteiro auxiliar para percorrer a lista sem alterar o início
```

```
    cout << "Elementos da lista: ";
```

```
    // Percorre a lista até atingir o fim (NULL)
```

```
    while (atual != NULL) {
```

```
        cout << atual->valor << " -> ";
```

```
        atual = atual->proximo; // Avança para o próximo nó
```

```
    }
```

```
    cout << "NULL" << endl;
```

```
}
```

```
int main() {  
    // Alocação e montagem de uma lista simples para teste: 10 -> 20 -> 30 -> NULL  
    No no3 = {30, NULL};  
    No no2 = {20, &no3};  
    No no1 = {10, &no2};  
  
    No* inicio = &no1;  
  
    // Chamada da função de percurso  
    percorrerLista(inicio);  
  
    return 0;  
}
```

19. Busca em lista Implemente a função `bool buscar(No* inicio, int valor)`. Ela deverá retornar `true` quando o elemento existir na lista e `false` quando não existir. Resposta / código:

```
#include <iostream>  
  
using namespace std;  
  
// Estrutura do Nó  
struct No {  
    int valor;  
    No* proximo;  
};
```

```
// Função para buscar um elemento na lista encadeada
```

```
bool buscar(No* inicio, int valor) {
```

```
    No* atual = inicio; // Ponteiro auxiliar para percorrer a lista
```

```
    // Percorre a lista até o final (NULL)
```

```
    while (atual != NULL) {
```

```
        if (atual->valor == valor) {
```

```
            return true; // Retorna true imediatamente ao encontrar o valor
```

```
        }
```

```
        atual = atual->proximo; // Avança para o próximo nó
```

```
    }
```

```
    return false; // Retorna false se percorrer toda a lista e não encontrar
```

```
}
```

```
int main() {
```

```
    // Criando uma lista para testes: 10 -> 25 -> 30 -> NULL
```

```
    No no3 = {30, NULL};
```

```
    No no2 = {25, &no3};
```

```
    No no1 = {10, &no2};
```

```
    No* inicio = &no1;
```

```
    // Teste 1: Buscar um valor que EXISTE na lista
```

```
    int valorProcurado1 = 25;
```

```
    if (buscar(inicio, valorProcurado1)) {
```

```
        cout << "O valor " << valorProcurado1 << " esta na lista!" << endl;
```

```
    } else {
```

```

        cout << "O valor " << valorProcurado1 << " nao foi encontrado." << endl;
    }

    // Teste 2: Buscar um valor que NÃO EXISTE na lista
    int valorProcurado2 = 99;
    if (buscar(inicio, valorProcurado2)) {
        cout << "O valor " << valorProcurado2 << " esta na lista!" << endl;
    } else {
        cout << "O valor " << valorProcurado2 << " nao foi encontrado." << endl;
    }

    return 0;
}

```

Parte IV – Desafios 20. Sistema de notas com vetor Uma turma possui 10 alunos. Desenvolva um programa que permita: cadastrar as 10 notas; apresentar todas as notas; calcular a média; apresentar a maior e a menor nota; informar quantos alunos ficaram acima da média; e permitir procurar uma determinada nota. [Escreva aqui o código]

```
#include <iostream>
```

```
using namespace std;
```

```

int main() {
    double notas[10];
    double soma = 0.0;

    // 1. Cadastrar as 10 notas
    cout << "=== Cadastro de Notas ===" << endl;
    for (int i = 0; i < 10; i++) {

```

```

        cout << "Digite a nota do " << i + 1 << "º aluno: ";

        cin >> notas[i];

        soma += notas[i];
    }

    // 2. Calcular a média da turma

    double media = soma / 10.0;

    // 3. Identificar maior, menor e contar notas acima da média

    double maior = notas[0];
    double menor = notas[0];
    int acimaDaMedia = 0;

    for (int i = 0; i < 10; i++) {
        if (notas[i] > maior) {
            maior = notas[i];
        }
        if (notas[i] < menor) {
            menor = notas[i];
        }
        if (notas[i] > media) {
            acimaDaMedia++;
        }
    }

    // Exibição do relatório de resultados

    cout << "\n=== Relatório Geral da Turma ===" << endl;

    // Apresentar todas as notas

```

```
cout << "Notas registradas: ";  
for (int i = 0; i < 10; i++) {  
    cout << "[" << notas[i] << "]" ";  
}  
cout << endl;
```

```
cout << "Media da turma: " << media << endl;  
cout << "Maior nota: " << maior << endl;  
cout << "Menor nota: " << menor << endl;  
cout << "Quantidade de alunos acima da media: " << acimaDaMedia << endl;
```

```
// 4. Permitir procurar uma determinada nota
```

```
double notaBusca;  
cout << "\n=== Busca de Nota ===" << endl;  
cout << "Digite a nota que deseja procurar: ";  
cin >> notaBusca;
```

```
bool encontrada = false;  
for (int i = 0; i < 10; i++) {  
    if (notas[i] == notaBusca) {  
        cout << "Nota " << notaBusca << " encontrada no aluno " << i + 1 << " (Indice:  
[" << i << "])" << endl;  
        encontrada = true;  
    }  
}
```

```
if (!encontrada) {  
    cout << "Nota nao encontrada na turma." << endl;  
}
```

```
    return 0;
}
```

21. Cadastro utilizando lista encadeada Desenvolva um programa utilizando lista encadeada que apresente o menu abaixo e execute as operações correspondentes: 1 - Inserir elemento 2 - Listar elementos 3 - Buscar elemento 4 - Remover elemento 5 - Contar elementos 0 – Sair

```
#include <iostream>
```

```
using namespace std;
```

```
// Estrutura do Nó (compatível com Dev-C++ / C++98)
```

```
struct No {
```

```
    int valor;
```

```
    No* proximo;
```

```
    No(int v, No* p = NULL) {
```

```
        valor = v;
```

```
        proximo = p;
```

```
    }
```

```
};
```

```
// 1. Inserir elemento no final da lista
```

```
void inserir(No*& inicio, int valor) {
```

```
    No* novo = new No(valor, NULL);
```

```
    if (inicio == NULL) {
```

```
        inicio = novo;
```

```
    } else {
```

```

    No* atual = inicio;

    while (atual->proximo != NULL) {
        atual = atual->proximo;
    }

    atual->proximo = novo;
}

cout << "Elemento " << valor << " inserido com sucesso!\n";
}

```

// 2. Listar todos os elementos

```

void listar(No* inicio) {
    if (inicio == NULL) {
        cout << "A lista esta vazia.\n";
        return;
    }

```

```

    cout << "Elementos na lista: ";

    No* atual = inicio;

    while (atual != NULL) {
        cout << atual->valor << " -> ";
        atual = atual->proximo;
    }

    cout << "NULL\n";
}

```

// 3. Buscar elemento na lista

```

void buscar(No* inicio, int valor) {
    No* atual = inicio;

    int posicao = 0;

```

```

while (atual != NULL) {
    if (atual->valor == valor) {
        cout << "Elemento " << valor << " encontrado no indice [" << posicao << "].\n";
        return;
    }
    atual = atual->proximo;
    posicao++;
}
cout << "Elemento " << valor << " nao encontrado na lista.\n";
}

```

// 4. Remover elemento da lista

```

void remover(No*& inicio, int valor) {
    if (inicio == NULL) {
        cout << "A lista esta vazia.\n";
        return;
    }
}

```

// Caso 1: O elemento a ser removido é o primeiro nó

```

if (inicio->valor == valor) {
    No* temp = inicio;
    inicio = inicio->proximo;
    delete temp;
    cout << "Elemento " << valor << " removido com sucesso!\n";
    return;
}

```

// Caso 2: Busca o nó anterior ao elemento que será removido

```

No* atual = inicio;

while (atual->proximo != NULL && atual->proximo->valor != valor) {
    atual = atual->proximo;
}

if (atual->proximo != NULL) {
    No* temp = atual->proximo;
    atual->proximo = temp->proximo;
    delete temp;
    cout << "Elemento " << valor << " removido com sucesso!\n";
} else {
    cout << "Elemento " << valor << " nao foi encontrado para remocao.\n";
}
}

```

// 5. Contar quantidade de elementos na lista

```

int contar(No* inicio) {
    int contador = 0;
    No* atual = inicio;
    while (atual != NULL) {
        contador++;
        atual = atual->proximo;
    }
    return contador;
}

```

// Função para liberar a memória alocada antes de sair

```

void limparLista(No*& inicio) {
    while (inicio != NULL) {

```

```

        No* temp = inicio;

        inicio = inicio->proximo;

        delete temp;
    }
}

int main() {
    No* inicio = NULL;

    int opcao, valor;

    do {
        cout << "\n===== \n";

        cout << "    MENU DE OPCOES    \n";

        cout << "===== \n";

        cout << "1 - Inserir elemento\n";
        cout << "2 - Listar elementos\n";
        cout << "3 - Buscar elemento\n";
        cout << "4 - Remover elemento\n";
        cout << "5 - Contar elementos\n";
        cout << "0 - Sair\n";

        cout << "Escolha uma opcao: ";

        cin >> opcao;

        switch (opcao) {
            case 1:
                cout << "Digite o valor a ser inserido: ";

                cin >> valor;

                inserir(inicio, valor);

                break;

```

```
case 2:
    listar(inicio);
    break;
case 3:
    cout << "Digite o valor a ser buscado: ";
    cin >> valor;
    buscar(inicio, valor);
    break;
case 4:
    cout << "Digite o valor a ser removido: ";
    cin >> valor;
    remover(inicio, valor);
    break;
case 5:
    cout << "Total de elementos na lista: " << contar(inicio) << endl;
    break;
case 0:
    cout << "Encerrando o programa e liberando memoria...\n";
    limparLista(inicio);
    break;
default:
    cout << "Opcao invalida! Tente novamente.\n";
}
} while (opcao != 0);

return 0;
}
```